

Ranveer Thakur Chand, John Nasir, Trevor Oakum, Freddy Rodriguez

Professor Viswanatha Rao

ITAI 1371

27 June 2026

Reflective Journal

Ranveer Thakur Chand:

In this lab, we focused on data preparation, which processes cleaning and transforming raw data before we input it into an ML model. We used the Titanic dataset from the previous lab, which according to the markdown cell in the lab was a good choice since it already had the exact issues we needed to work through, like missing values and non-numeric columns. In the first part, we were to handle missing values through imputation. The Age column had 177 missing entries, so instead of dropping those rows, I filled them using the median values. We used the median because it works better than the mean in this case since the age data is skewed by outliers like we saw in the previous lab from the graphs, and the median is less sensitive to those extremes. In the `print(df.isnull().sum())` you could see that the Cabin column had even more missing values but wasn't used in this lab since we dropped it in the previous lab because of too many values missing. We also did the same with Embarked through encoding. I was confused why we included embarked since it has 3 columns but then I understood that it included all three but it showed as 2 because we dropped one redundant column. In the second part was one-hot encoding the Sex and Embarked columns. Since ML models only work with numbers, text categories like "male" or "S" need to be converted with `pd.get_dummies()`. This function is a built-in panda tool which makes it easy to create new binary columns for each category. Using `drop_first=True` removed one redundant column per group, which prevents a statistical issue called multicollinearity. Finally we used

feature scaling using `StandardScaler()` on the Age and Fare columns. These two features had very different ranges. Age went from 0 - 80, while fare could go from 0 – 500+. Without scaling, the model might weigh fare more heavily simply because the numbers are bigger, not because it's more important. Standardization brought both columns to a mean of 0 and a standard deviation of 1. Overall, this lab was a pretty quick and practical walkthrough of three core preprocessing steps.

John Nasir:

This lab was the first time I really saw how much of machine learning happens before you train. Working through the notebook made the saying “Garbage in, garbage out” is true. Even small bad things in raw data, makes it hard for a model’s understanding. When i saw ranveer make edits, and fixing those issues helped changed how the dataset behaved and how meaningful the patterns became.

Handling missing data was where I learned. Before this lab, I didn’t realize there were different types of missingness (MCAR, MAR, MNAR) or that even each one requires a different strategy. Using simple imputation (mean, median, mode) felt easy, but i saw how KNN imputation used similarity between rows and it made me understand why advanced methods can have more structure. Now i also know why blindly dropping rows can be dangerous.

Encoding categorical variables was another big takeaway. This lab showed me how badly things can go if you use the wrong one. Label encoding only works when the categories have a real order, and if the team does it wrong it explodes, especially if the feature has too many unique values. Seeing this helped me understand why high-cardinality features need special handling like grouping or target encoding.

In scaling, I didn’t realize how much algorithms like KNN or SVM depend on feature scale. After Ranveer applied `StandardScaler` and `MinMaxScaler`, I could see how the numbers changed and why scaling

prevents certain features from dominating others. The “prevent leakage” rule stood out to me fitting the scaler only on the training set is something I’ll remember for future labs.

Feature engineering was probably the most interesting part. The features captured real-world meaning that wasn’t obvious in the raw data. It showed me how domain knowledge can improve accuracy more than just picking a algorithm.

Data preparation is the foundation of machine learning. Clean, well-structured data makes everything else easier, and I now see why professionals spend most of their time on this stage. I feel more confident going into future labs because I understand how to transform messy data into something a model can actually learn from.

Trevor Oakum:

This lab both taught me new things about how algorithms learn data, and reinforced some ideas I knew from statistics classes in previous years. In Statistics, anytime data is missing, it’s always important to try to make sure that the missing data is found in the correct manner to avoid issues. Applying this logic to an algorithm learning, and it would be catastrophic to insert data incorrectly, as not only has the data been compromised, but now, a chain reaction has spread to where the model has now had the opportunity taken the data as fact, the data is interpreted correctly, and now the model might be stubborn to program new data. Because of this knowledge I had previously in other classes about statistics, and the importance of collecting data not only correctly, but making sure it's implemented correctly, I learned what happens when models insert data correctly. I had never used this much raw data and fed it to an algorithm, but I have fed and compiled raw data to generate results (although nowhere near this much data, and data that is more simple to understand).

When doing this lab, I was also trying to be as careful as possible to make sure the data wasn't corrupted after data (even if it was correctly collected), as this can happen from time to time, mainly due to errors in performing necessary functions. This made me think further of how could data be collected in better ways in the future, and how many more ways can one collect data that an algorithm could visualize data, or even further, what other ways could a model collect that we have yet to understand, ways in which we dismissed previously, or ways to collect data we have never seen before. I am curious to see where this could go with the project.

Freddy Rodriguez:

The Language Barrier Breakthrough (One-Hot Encoding) Think about your favorite AI tools or gaming algorithms. They seem incredibly smart, right? But here's the secret: at their core, AI models are just massive, glorified calculators. They only understand pure math. If you type in the words "Male," "Female," or a location like "Cherbourg," the computer doesn't get a mental image. It doesn't know what a gender is, and it definitely doesn't know that Cherbourg is a port city in France. To the AI, those words are just meaningless groups of letters. It's totally illiterate to the human world. To fix this, we use a trick called One-Hot Encoding. Think of it like a multiple-choice survey with a bunch of checkboxes, where you can only pick one answer. Instead of forcing a math equation to read the word "Cherbourg," we break it down into a simple game of 0 (No) and 1 (Yes). If the AI needs to know where a Titanic passenger got on the ship, we give it three columns: [Cherbourg] [Queenstown] [Southampton] If they boarded at Cherbourg, the data looks like this: [1] [0] [0] If they boarded at Southampton, it looks like this: [0] [0] [1] We realized that machine learning models are essentially massive calculators. They don't know what 'male', 'female', or 'Cherbourg' mean. One-Hot Encoding acts as the ultimate translator. By

breaking text down into columns of 1s and 0s, we literally translated human labels into binary logic that math can understand.